

The tools registry and built-in actions

Week 5 Lecture 1 | Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- The Tools registry: what it stores and how it is populated
- Built-in actions: the ten you get without writing any code
- ActionModel: how registered actions become the structured choice
- ActionResult: what comes back and where it goes
- How action descriptions reach the LLM system prompt

The registry

The Tools registry

The `Tools` object holds an `ActionRegistry`.

The registry maps each action name to a `RegisteredAction` object.

Each `RegisteredAction` stores:

- **name**: the key the LLM writes
- **description**: what the LLM reads to decide when to call it
- **handler function**: the Python callable that runs
- **param_model**: a Pydantic model for the parameters

(browser-use contributors, 2026, `browser_use/tools/registry/views.py`)

From registry to ActionModel

The LLM does not pick from a free list of strings.

The registry's registered actions are compiled into an **ActionModel**: a dynamically built Pydantic model where each action is an optional field.

The LLM's reply is validated against ActionModel.

- Unknown action name → validation fails
- Wrong parameter type → validation fails
- Valid reply → dispatch proceeds

How descriptions reach the LLM

```
action_name: description. (param1=type, param2=type, ...)
```

`registry.get_prompt_description()` generates this text.

- No URL provided (system prompt): returns actions with no domain filter
- URL provided (per-step context): also includes domain-matched actions

The description field is required and is the only signal the LLM has for deciding when to call the action.

(browser-use team, 2026, docs: Custom Functions)

Built-in actions

The action vocabulary (1 of 2)

Action	Key parameters	terminates_sequence
click	index: int	No
input_text	index: int, text: str	No
scroll	index, direction, amount	No
go_to_url	url: str	Yes
go_back	(none)	Yes

The agent refers to elements by integer index from the selector map (week 2), not by CSS selector.

The action vocabulary (2 of 2)

Action	Key parameters	terminates_sequence
search_google	query: str	Yes
extract_content	goal: str	No
switch_tab	tab_id: int	No
open_tab	url: str	No
done	text: str, success: bool	N/A

`done` is the stopping signal. The loop exits when `ActionResult.is_done=True`.

ActionResult

What an action returns

```
class ActionResult(BaseModel):  
    is_done: bool | None = False  
    success: bool | None = None  
    error: str | None = None  
    extracted_content: str | None = None  
    long_term_memory: str | None = None  
    include_in_memory: bool = False
```

(browser-use contributors, 2026, `browser_use/agent/views.py`)

`success=True` can only be set when `is_done=True`.

Where the result goes

1. Handler returns `ActionResult`
2. Dispatch layer collects all results for the step
3. `extracted_content` appears in the **next step's prompt**
4. The LLM writes `evaluation_previous_goal` based on that content
5. Errors count against `max_failures`

The loop is a closed feedback cycle: every action's result shapes the next decision.

Take-aways

1. The `Tools` registry maps names to `RegisteredAction` objects: name, description, handler, param_model.
2. Registered actions become the `ActionModel` the LLM is validated against: unknown names fail immediately.
3. `get_prompt_description()` puts action descriptions in the system prompt; vague descriptions produce wrong choices.
4. Ten built-in actions cover the full lifecycle: navigate, interact, extract, and done.
5. `ActionResult` carries extracted content and errors back to the loop; the next step's prompt contains that content.

"The LLM uses this to decide when to call it."

What is next

- **Lecture 2:** multi_act, the terminates_sequence abort rule, and parameter injection by name
- **Reading:** The tools registry and dispatch: `/c/browser-use-26su/readings/wk05`
- **Section:** Follow one click from AgentOutput to the page and watch the result return to the loop